

Требования к данному заданию:

- мы принимаем готовое тестовое задание только в формате ссылки на GitHub
- быть готовым на собеседовании ответить на любой вопрос по отправленному решению
- права на редактирование данного задания получить нельзя, запрос отправлять бессмысленно
- в конце заданию даны рекомендации для подготовки к собеседованию

Блок 1: Теория вероятности и логика

Задание 1: Фермер

На ферме содержатся шесть разных видов животных, и каждый раз, когда фермер заходит в сарай, он видит одно случайное животное. За день фермер заходит в сарай 6 раз.

Каково математическое ожидание количества разных видов животных, которые фермер увидит за день?

Ответ (округлить до сотых): **3.99**

Задание 2: Кулинарное соревнование

В конкурсе участвуют 80 шеф-поваров с уникальными уровнями мастерства. В первом этапе судьи случайным образом распределяют их по парам (в любом состязании двух шефов выигрывает тот, у кого выше уровень мастерства). На втором этапе шефы снова случайно образуют пары для финального раунда (пары могут повториться). Победную награду получают те, кто выиграл в обоих этапах.

Каково математическое ожидание числа победителей?

Ответ (округлить до десятых): **26.8**

Задание 3: Одинокая дорога

На пустынном шоссе вероятность появления автомобиля за 30-минутный период составляет 0.95.

Какова вероятность его появления за 10 минут? А за 27 минут?

Ответ (дать в процентах, округлив до десятых через точку с запятой): **63.2; 93.3**

Блок 2: Python

Задание 1: Изоморфизмы

Реализовать функцию (или тело функции), которая проверяет на изоморфность два слова. Пояснение: строки *s* и *t* называются изоморфными, если все вхождения каждого символа строки *s* можно последовательно заменить другим символом и получить строку *t*. Порядок символов при этом должен сохраняться, а замена — быть уникальной. Так, два разных символа строки *s* нельзя заменить одним и тем же символом из строки *t*, а вот одинаковые символы в строке *s* должны заменяться одним и тем же символом.

```
# Пример:
s = 'paper'
t = 'title'
print(is_isomorphic(s, t))
# Вывод:
True
```

Оценить оптимальность решения по времени и памяти и прикрепить текст кода.

def is_isomorphic(s: str, t: str) -> bool:

```
    if len(s) != len(t):
```

```
        return False
```

```
    def equal(sym: str):
```

```
        d = {}
```

```
        lst = []
```

```
        for i, s in enumerate(sym):
```

```
            if s not in d:
```

```
                d[s] = i
```

```
            lst.append(d[s])
```

```
        return lst
```

```
    return equal(s) == equal(t)
```

```
s = 'paper'
```

```
t = 'title'
```

```
print(is_isomorphic(s, t))
```

Общая временная сложность: $O(n)$, где n = длина строк, Память: $2 * O(n) = O(n)$

Задание 2: Натуральная последовательность

Реализовать функцию (или тело функции), которая находит единственное отсутствующее число из последовательности натуральных чисел $1, 2, \dots, n$.

```
# Пример:
nums = [1, 2, 3, 4, 5, 6, 8, 9, 10, 11]
print(missing_number(nums))
# Вывод:
7
```

Оценить оптимальность решения по времени и памяти и прикрепить текст кода.

```
def missing_number(nums: list) -> int:
```

```
    for i in range(1, len(nums) + 2):
```

```
        if i not in nums:
```

```
            return i
```

Квадратичное время $O(n^2)$ - неоптимально для этой задачи

Лучшее решение:

```
def missing_number(nums: list) -> int:
```

```
    n = len(nums) + 1
```

```
    return n * (n + 1) // 2 - sum(nums)
```

Время $O(n)$ - оптимально (нужно посмотреть все элементы)

Память $O(1)$ - оптимально (только несколько переменных)

Задание 3: Факторизация

Реализовать функцию (или тело функции), которая при введении натурального числа n разбивает его на простые множители (представить его в виде простых чисел).

```
# Пример:
n = 56
print(prime_factors(n))
# Вывод:
[2, 2, 2, 7]
```

Оценить оптимальность решения по времени и памяти и прикрепить текст кода.

```

def prime_factors(n: int) -> list:
    factors = []
    # обрабатываем делитель 2 отдельно
    while n % 2 == 0:
        factors.append(2)
        n //= 2
    # теперь проверяем только нечётные делители
    d = 3
    while d * d <= n:
        while n % d == 0:
            factors.append(d)
            n //= d
        d += 2 # только нечётные
    if n > 1:
        factors.append(n)
    return factors

```

Общая временная сложность: $O(\sqrt{n})$, Память $O(\log n)$ - храним только результат

Блок 3: SQL

Задание 1: Абитуриенты

Есть таблица `examination` с двумя полями: `id` (id абитуриента), `scores` (кол-во набранных баллов дополнительного вступительного испытания от 0 до 100).

Требуется реализовать запрос, который создаёт колонку с позицией абитуриента в общем рейтинге.

В зависимости от того какой рейтинг мы хотим увидеть (с пропуском ранга или без) будет реализована оконная функция **DENSE_RANK()** или **DENSE()**, можно также использовать **ROW_NUMBER()**

```

SELECT
    id,
    scores,
    DENSE_RANK() OVER (ORDER BY scores DESC) AS position
FROM table
ORDER BY —сортировка
LIMIT —ограничение количества выводимых строк
OFFSET;

```

Задание 2: FULL JOIN

Представьте две таблицы: первая содержит 30 строк, а вторая — 20 строк. Мы выполняем операцию FULL JOIN между ними.

Какой диапазон возможного количества строк может быть в результирующей таблице, если учесть, что ключи для соединения могут быть как полностью совпадающими, так и абсолютно уникальными?

Ответ дать в краткой форме, например: минимально 10 и максимально 3000 строк

ключи совпадающие: минимально 30 и максимально 50

ключи уникальные: у обеих таблиц будет 50 строк

Задание 3: Покупки

```
create table account
(
    id integer, -- ID счета
    client_id integer, -- ID клиента
    open_dt date, -- дата открытия счета
    close_dt date -- дата закрытия счета
)

create table transaction
(
    id integer, -- ID транзакции
    account_id integer, -- ID счета
    transaction_date date, -- дата транзакции
    amount numeric(10,2), -- сумма транзакции
    type varchar(3) -- тип транзакции
)
```

Вывести ID клиентов, которые за последний месяц по всем своим счетам совершили покупок меньше, чем на 5000 рублей.

Без использования подзапросов и оконных функций.

```
WITH purchase AS (
    SELECT t.account_id AS client,
           sum(t.amount)
```

```

FROM transaction t
JOIN account a ON t.account_id = a.client_id
WHERE t.TYPE = 'PUR'
GROUP BY t.account_id
HAVING sum(t.amount) < 50000
)
SELECT client
FROM purchase
ORDER BY client;

```

Блок 4: Статистика и АБ-тесты

Задание 1: Воодушевленное руководство

Вы – аналитик компании Самокат (сервис по доставке продуктов на дом).

Команда решила протестировать гениальную идею замены транспортного средства доставщиков и провели АБ эксперимент в небольшом городке РФ. Результаты превзошли все ожидания: время доставки значительно снизилось в несколько раз! Руководству не терпится применить изменения по всей стране.

Делаем? Выберите все верные утверждения:

- ☐ Делаем. Только применяем изменения в таком же масштабе (количество транспортных средств с изменением), как в городе, где проводили тест.
- ☒ **Тест нерепрезентативен, поэтому результаты применять по всей стране нельзя.**
- ☐ Тест репрезентативен относительно своей генеральной совокупности: таких же небольших городов, можем применять только в подобных городах.
- ☒ **Наличие эффекта подтвердило потенциал идеи, поэтому сразу применять по всем городам не будем, но можем провести эксперимент в других городах.**

Задание 2: Основной показатель в статистике

Что такое p-value в статистическом тестировании? Выберите одно верное утверждение:

- ☐ Вероятность, что нулевая гипотеза верна.
- ☒ **Вероятность наблюдения такого или более экстремального результата при условии, что нулевая гипотеза верна.**
- ☐ Значение уровня значимости, при котором отвергается нулевая гипотеза.
- ☐ Среднее значение выборки.

Задание 3: Параметрический тест

Вам необходимо провести аб-тестирование, целевая метрика - среднее число уникальных покупок на пользователя. Размер выборки велик (более 5 млн наблюдений в группах теста и контроля), значительные выбросы в выборках отсутствуют.

Можем ли мы применить t-критерий Стьюдента для проверки гипотезы о неравенстве средних в тестовой и контрольной группах при условии, что распределение уникальных покупок является логнормальным?

Ответ: Нет, нельзя применять t-критерий Стьюдента напрямую

Причины:

- Т-критерий Стьюдента предполагает нормальное распределение данных в группах;

- Логнормальное распределение сильно асимметрично (имеет длинный правый хвост), это нарушает предположение о нормальности остатков;

Проблемы с логнормальным распределением:

- Среднее значение в логнормальном распределении смещено вправо

- Т-критерий может давать ложные результаты при асимметричных распределениях

- Доверительные интервалы будут некорректными

Что делать вместо этого:

Логарифмическое преобразование данных:

python

```
# Если данные логнормальны, то их логарифмы распределены нормально
```

```
log_data_test = np.log(test_data)
```

```
log_data_control = np.log(control_data)
```

```
# Затем можно применять t-критерий к логарифмированным данным
```

Непараметрические альтернативы:

U-критерий Манна-Уитни (для независимых выборок)

При больших выборках (>5 млн):

Можно использовать z-тест (центральная предельная теорема гарантирует нормальность выборочных средних)

Почему z-тест может работать:

При $N > 5$ млн:

Распределение выборочного среднего будет приближенно нормальным (ЦПТ)

Стандартные ошибки будут очень малы

Но нужно проверять однородность дисперсий!

Что можно сделать:

Использовать логарифмическое преобразование + t-критерий

Или напрямую применить U-критерий Манна-Уитни

Или использовать z-тест с проверкой гомогенности дисперсий

Важно: Даже при больших выборках форма распределения исходных данных влияет на мощность теста и интерпретацию результатов!

Блок 5: ML Base

Задание 1: Пони тоже кони

Вас просят разработать модель, классифицирующую лошадок и пони. Вместо разработки вы нашли на GitHub две интересные модели и после прогона на ваших данных одна из них показала ROC-AUC=0.7, а другая ROC-AUC=0.1. Какую модель вы возьмете для дальнейшей работы и что будете с ней делать?

Ответ написать развернутый, но кратко, например: "первую, отниму от метрики 0.1 для корректировки точности": **Выбираем первую модель с некоторыми предсказательными способностями (случайное угадывание = 0,5). Диагностика текущего состояния:**

Посмотреть confusion matrix

Проверить precision, recall, F1-score

Проанализировать ошибки: какие примеры классифицируются неправильно

Исследование данных:

Проверить баланс классов (лошади vs пони)

Анализ качества и репрезентативности данных

Выявление потенциальных проблем в данных

Улучшение модели (ROC-AUC = 0.7 → целевой 0.85+):

Подбор гиперпараметров, + другие алгоритмы

Задание 2: Ручной счёт ROC_AUC

Классификатор выдал следующие прогнозируемые метки класса и вероятности принадлежности к классу "1". На основе полученных данных рассчитайте метрику ROC_AUC. Тезисно описать ход решения.

Ответ округлить до сотых, например: 4,12

Истинная метка класса	Порог классификации (0.6)	Оценка вероятности
1	1	0.95

0	1	0.9
1	1	0.85
0	1	0.8
1	1	0.75
1	1	0.7
1	1	0.65
1	1	0.6
0	0	0.55
0	0	0.5
0	0	0.45
1	0	0.4
0	0	0.35
0	0	0.3
0	0	0.25

Подсчет:

Объектов класса 1: 7 (индексы: 0, 2, 4, 5, 6, 7, 11)

Объектов класса 0: 8 (индексы: 1, 3, 8, 9, 10, 12, 13, 14)

Всего возможных пар: $7 \times 8 = 56$

Для каждого объекта класса 1 считаем, сколько объектов класса 0 имеют меньшую вероятность:

Объект 1 (вероятность 0.95, класс 1):

Все 8 объектов класса 0 имеют меньшую вероятность ✓

Объект 3 (вероятность 0.85, класс 1):

Объекты класса 0 с меньшей вероятностью: все, кроме первого (0.9) → 7 ✓

Объект 5 (вероятность 0.75, класс 1):

Объекты класса 0 с меньшей вероятностью: все, кроме первых двух (0.9, 0.8) → 6 ✓

Объект 6 (вероятность 0.7, класс 1):

Объекты класса 0 с меньшей вероятностью: кроме (0.9, 0.8) → 6 ✓

Объект 7 (вероятность 0.65, класс 1):

Объекты класса 0 с меньшей вероятностью: кроме (0.9, 0.8) → 6 ✓

Объект 8 (вероятность 0.6, класс 1):

Объекты класса 0 с меньшей вероятностью: кроме (0.9, 0.8, 0.55) → 5 ✓

Объект 12 (вероятность 0.4, класс 1):

Объекты класса 0 с меньшей вероятностью: кроме (0.9, 0.8, 0.55, 0.5, 0.45, 0.35, 0.3, 0.25) → 0

Сумма правильных пар: $8 + 7 + 6 + 6 + 6 + 5 + 0 = 38$

ROC-AUC = $38 / 56 \approx 0.68$

Задание 3: Ручной счёт корреляции

Рассчитайте линейную корреляцию Пирсона, на основе данных.

Какой вывод можно сделать на основе полученного результата? Можно ли утверждать, что существует причинно-следственная связь между количеством чашек кофе, выпитых студентами в течение экзаменационного дня, и их итоговым баллом за экзамен?

Ответ округлить до сотых, например: 4,12

Число выпитых чашек кофе	Балл за экзамен
1	85
1	88
2	79
2	81
2	84
2	65
3	67
3	58
3	76
4	49

$$r = \frac{n\sum XY - \sum X \sum Y}{\sqrt{[n\sum X^2 - (\sum X)^2][n\sum Y^2 - (\sum Y)^2]}}$$

1. Данные:

X (чашки кофе): [1, 1, 2, 2, 2, 2, 3, 3, 3, 4]

Y (баллы): [85, 88, 79, 81, 84, 65, 67, 58, 76, 49]

n = 10

2. Вычисление сумм:

$\sum X = 1+1+2+2+2+2+3+3+3+4 = 23$

$\sum Y = 85+88+79+81+84+65+67+58+76+49 = 732$

$\sum X^2 = 1+1+4+4+4+4+9+9+9+16 = 61$

$$\Sigma Y^2 = 7225 + 7744 + 6241 + 6561 + 7056 + 4225 + 4489 + 3364 + 5776 + 2401 = 55082$$

$$\begin{aligned}\Sigma XY &= (1 \times 85) + (1 \times 88) + (2 \times 79) + (2 \times 81) + (2 \times 84) + (2 \times 65) + (3 \times 67) + (3 \times 58) + (3 \times 76) + (4 \times 49) \\ &= 85 + 88 + 158 + 162 + 168 + 130 + 201 + 174 + 228 + 196 = 1590\end{aligned}$$

Числитель:

$$10 \times 1590 - 23 \times 732 = 15900 - 16836 = -936$$

Знаменатель:

$$\text{Часть 1: } n\Sigma X^2 - (\Sigma X)^2 = 10 \times 61 - 23^2 = 610 - 529 = 81$$

$$\text{Часть 2: } n\Sigma Y^2 - (\Sigma Y)^2 = 10 \times 55082 - 732^2 = 550820 - 535824 = 14996$$

$$R = -936 / \sqrt{1,214,676} = \mathbf{-0.85}$$

Корреляция отрицательная, сильная, вывод: чем больше выпито чашек кофе, - тем хуже результат на экзамене.

Рекомендации по подготовке к собеседованию в случае успешного решения тестового задания

- Знать основы теории вероятностей;
- Знать основы математической статистики;
- Иметь практический навык реализации базовых запросов на SQL;
- Знать основы реализации алгоритмических задач на Python и оценки сложности алгоритмов;
- Приветствуется коммерческий опыт и знания в области планирования и проведения АБ-тестов.